SmartCampus — Dossier final de synthèse

Architecture de données, modélisation et résilience IoT

Nature du document : synthèse individuelle des étapes 1 à 8 du projet
Périmètre : supervision technique et énergétique d'un campus
Statut : dossier de conception — les hypothèses, propositions et décisions à valider sont signalées comme telles

---

Résumé exécutif

SmartCampus est une plateforme de supervision d'infrastructures techniques et énergétiques. Elle doit relier le référentiel spatial et technique aux mesures produites par les capteurs, aux alertes et aux interventions de maintenance, tout en conservant le contexte temporel des événements.

Le dimensionnement pédagogique de référence est de 8 bâtiments, environ 160 zones, 250 équipements, 600 capteurs et 50 utilisateurs actifs en journée. À raison d'une mesure par capteur toutes les 30 secondes, la charge nominale calculée est de **20 mesures par seconde**, **1 728 000 par jour** et **630 720 000 par an** sur 365 jours, avant retransmissions. La rétention brute cible est de 12 mois et les tableaux de bord sont actualisés toutes les 5 minutes. Ces valeurs sont des hypothèses de dimensionnement, pas des mesures de production.

La recommandation est de démarrer avec **PostgreSQL comme source de vérité unique**, un référentiel normalisé, des associations temporelles explicites, et une table de mesures partitionnée mensuellement. L'ingestion doit supporter les doublons, les retards, les rafales et les interruptions réseau au moyen d'identifiants de messages définis, de traitements idempotents, de files ou tampons durables et de procédures de rejeu. Les projections de lecture et les moteurs complémentaires restent conditionnels à des besoins quantifiés et à un benchmark reproductible.

Le dossier décrit un modèle cohérent de bout en bout, mais ne prétend pas valider des performances, un SLA ou un DDL exécuté. Les durées de conservation autres que celle des mesures brutes, les contrats d'identification des messages, les règles de qualité/unité, les transitions métier et les objectifs RPO/RTO doivent encore être approuvés.

1. Contexte métier et hypothèses de dimensionnement

1.1 Périmètre fonctionnel

La plateforme suit un ou plusieurs campus, leurs bâtiments et zones, les équipements qui y sont installés, les capteurs qui leur sont affectés, les mesures observées, les règles de détection, les alertes et les interventions de maintenance. Les usagers consultent l'état courant et l'historique; les administrateurs maintiennent le référentiel et les règles; les techniciens réalisent les interventions; les responsables énergie analysent les consommations.

1.2 Hypothèses quantitatives de référence

Paramètre	Valeur de travail	Statut / commentaire
Bâtiments	8	Valeur du cas pratique
Zones	Environ 160	Ordre de grandeur, non contrainte d'intégrité
Équipements	250	Population de référence
Capteurs	600	Population de référence
Périodicité	1 mesure / 30 s / capteur	Cadence nominale
Débit nominal global	20 mesures/s	Calculé : 600 / 30
Mesures/jour	1 728 000	Calcul : 20 × 86 400
Mesures/an	630 720 000	Calcul sur 365 jours, avant doublons

Paramètre	Valeur de travail	Statut / commentaire
Rétention brute	12 mois	Cible fournie par le cas; politique opérationnelle à préciser
Utilisateurs actifs	Environ 50	Charge diurne indicative
Actualisation des tableaux de bord	Toutes les 5 minutes	Cadence d'actualisation, pas un SLO de latence

Une rafale synchronisée de référence peut correspondre à 600 événements reçus simultanément toutes les 30 secondes, sans modifier la moyenne nominale. Une reconnexion après une coupure plus longue doit être testée comme un scénario distinct, avec un backlog explicitement paramétré.

1.3 Acteurs

Acteur	Objectifs et usages	Principales données lues	Données créées ou modifiées
Exploitant	Surveiller les équipements, qualifier un incident et solliciter une intervention.	État courant, mesures et historique utile, localisation, alertes, interventions.	Demandes d'intervention, commentaires et contexte d'exploitation.
Technicien	Diagnostiquer et exécuter des interventions, documenter le résultat.	Interventions affectées, équipement, emplacement applicable, historique et alertes associés.	Prise en charge, statut, dates, compte rendu et observations.
Responsable énergie	Suivre les consommations et comparer les tendances par période et périmètre.	Mesures, agrégats, unités, bâtiments, zones et équipements.	Paramètres de rapports ou annotations si cette fonction est retenue.
Administrateur	Maintenir le référentiel, les comptes, les droits et les règles.	Référentiel complet, configuration, événements d'administration.	Bâtiments, zones, équipements, capteurs, affectations, règles, comptes et habilitations.
Capteurs / passerelles IoT	Produire et transmettre des observations horodatées, éventuellement après mise en tampon.	Configuration de transmission et accusés de réception.	Mesures, identifiant source, horodatage d'événement et informations de qualité disponibles.

Les capteurs et passerelles ont une identité technique propre; ils ne sont pas des utilisateurs humains. L'authentification, les rôles et les autorisations sont à définir dans le contrat de sécurité de l'API IoT et de l'application.

1.4 Besoins fonctionnels et contraintes

1. **Référentiel** : représenter les liens campus–bâtiment–zone et équipement–capteur avec des identifiants stables et des contrôles d'unicité.
2. **Ingestion IoT** : recevoir les valeurs, unités, horodatages et indicateurs de qualité; tolérer les reprises, répétitions et données retardées.
3. **État courant** : exposer la dernière mesure reçue et, séparément, la dernière mesure valide, avec sa fraîcheur.
4. **Historique** : permettre les consultations temporelles et la restitution de l'emplacement ou de l'affectation en vigueur à la date d'un événement.
5. **Alertes et maintenance** : conserver la règle/version, la cible, la mesure déclencheuse éventuelle et la traçabilité des interventions multi-techniciens.
6. **Analytique** : définir unités, conversions, granularités, agrégations et traitement des mesures manquantes/invalides avant de comparer des consommations.
7. **Exploitation** : superviser débit, retards, erreurs, backlog, espace, partitions, sauvegardes et accès.

2. Étape 1 — Règles de gestion et exigences

2.1 Règles de gestion retenues pour le modèle de travail

ID	Règle de gestion	Statut / conséquence
RG-01	Un campus regroupe des bâtiments; chaque bâtiment dépend d'un seul campus.	Cardinalités retenues ci-après; nombre de campus non fourni.
RG-02	Un bâtiment contient des zones; chaque zone appartient à un seul bâtiment.	Environ 20 zones par bâtiment en moyenne n'est pas une contrainte fixe.
RG-03	Un équipement change éventuellement de zone au cours du temps.	Localisation portée par une association historisée; une seule localisation à un instant donné est le choix de travail.
RG-04	Un capteur peut être sans affectation; à un instant donné, il est affecté à zéro ou un équipement.	Plusieurs capteurs peuvent superviser un même équipement; les affectations sont historisées.
RG-05	Une mesure provient d'un seul capteur et comporte au minimum un instant d'événement, une valeur et une unité.	L'horodatage de réception est distinct; l'identifiant de message source reste à contractualiser.
RG-06	Une mesure peut être valide, invalide ou estimée selon le vocabulaire défini par type de capteur.	Les codes de qualité et les règles de rejet ne sont pas encore spécifiés.
RG-07	Une alerte est produite par une version déterminée d'une règle et concerne exactement un équipement ou un capteur.	Contrainte XOR; la cible doit être cohérente avec la portée de la règle.
RG-08	Une alerte peut être reliée à zéro, une ou plusieurs interventions; une intervention peut traiter plusieurs alertes.	Association plusieurs-à-plusieurs explicite.
RG-09	Une intervention porte sur un équipement et peut avoir un demandeur humain optionnel.	Les interventions automatiques/préventives peuvent ne pas avoir de demandeur.
RG-10	Une intervention peut mobiliser plusieurs techniciens, et un technicien peut participer à plusieurs interventions.	L'affectation de technicien est historisée et porte un rôle éventuel.
RG-11	Les codes métier et identifiants externes sont uniques dans leur périmètre défini.	Courriel, matricule, référence d'intervention et identifiant externe capteur sont candidats à l'unicité; normalisation de casse à décider.
RG-12	Les mesures brutes sont conservées pendant 12 mois, selon une politique de rétention à mettre en œuvre.	Ne détermine pas automatiquement la rétention des alertes, interventions, historiques et preuves.

2.2 Exigences non fonctionnelles

- **Durabilité** : un accusé de réception ne doit être émis qu'après dépôt durable au niveau défini dans l'architecture.
- **Idempotence** : les redélivrances ne doivent pas créer de mesures ou alertes dupliquées; une clé fiable et son périmètre sont nécessaires.
- **Temps** : conserver l'heure de l'événement et l'heure de réception séparément; les instants sont gérés en UTC.
- **Performance** : indexer les parcours réellement attendus et les confirmer par plans d'exécution et benchmark.
- **Disponibilité et reprise** : RPO/RTO, disponibilité cible et perte acceptable doivent être approuvés; les propositions de ce document ne sont pas des engagements.
- **Sécurité** : authentifier les sources, contrôler l'accès par rôle/périmètre, auditer les changements sensibles et protéger les sauvegardes.
- **Exploitabilité** : automatiser et surveiller les partitions, la rétention, les reprises et la restauration.

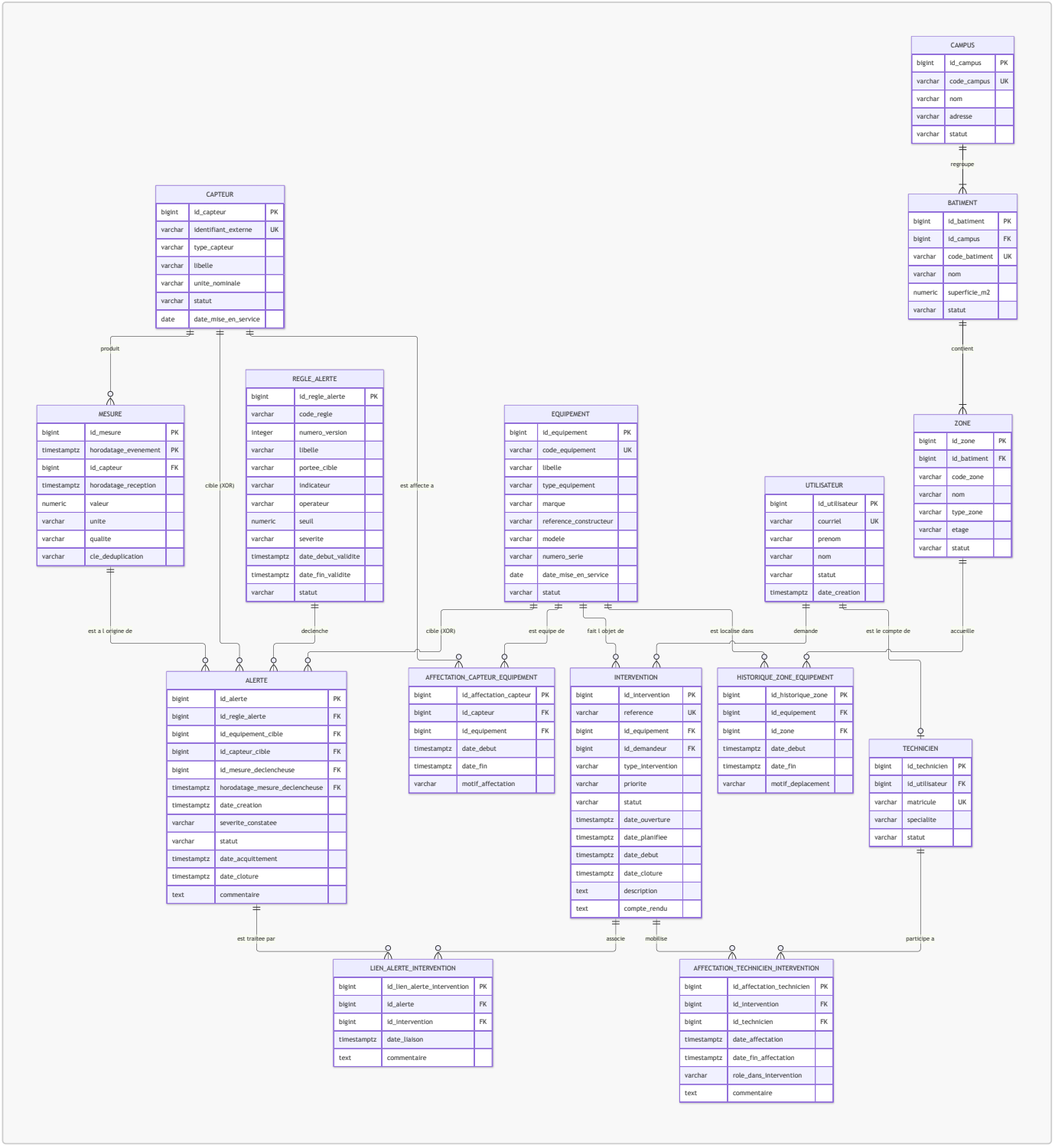
2.3 Zones grises à arbitrer

Sujet	Hypothèse de conception	Décision nécessaire
Création d'un campus/bâtiment sans enfant	Chaque bâtiment est rattaché à un campus; une zone à un bâtiment.	Autoriser ou non des états transitoires sans enfant ou sans rattachement.
Localisation d'un équipement	Historique d'intervalles sans chevauchement; zéro localisation avant installation ou après retrait.	Confirmer si un équipement doit toujours avoir une zone lorsqu'il est en service et autoriser les corrections rétroactives.
Capteur sans équipement	Autorisé avant installation, après retrait ou pour un capteur autonome.	Confirmer les cas de capteurs de zone/ambiance et les capteurs partagés.
Mesures sans affectation active	Mesure conservée avec son capteur; association temporelle résolue lors d'une consultation si possible.	Définir rejet, quarantaine ou conservation lorsqu'un capteur inconnu/inactif émet.
Horloges et ordre des événements	`horodatage_evenement` décrit l'observation; `horodatage_reception` décrit l'arrivée.	Préciser synchronisation, tolérance de dérive, séquence source et politique d'événements futurs/anciens.
Déduplication	Une clé source est préférée; absence de clé n'autorise pas à dédupliquer par simple couple capteur/horodatage.	Définir portée d'unicité, durée de conservation des clés et comportement en collision.
Qualité, unités et types	Unité présente sur chaque mesure, unité nominale dans le capteur.	Définir vocabulaires, précision, conversions et règles d'agrégation.
Cycle d'alerte	Règle versionnée, statut, acquittement et clôture.	Définir hystérésis, répétition, regroupement, escalade, acquittement et réouverture.
Demandeur et cible de maintenance	Demandeur optionnel; intervention sur un équipement.	Décider si une intervention peut viser directement une alerte/capteur/zone sans équipement.
Rétention et preuve	12 mois concerne les mesures brutes.	Définir conservation d'alertes/preuves et éviter qu'une FK bloque la purge de partitions de mesures.
SLA / RPO / RTO	Valeurs proposées plus loin à titre de travail seulement.	Faire approuver les cibles par métier, sécurité et exploitation.

3. Étape 2 — Modèle Conceptuel de Données

3.1 Diagramme conceptuel visuel (MCD)

Le diagramme entité-relation ci-dessous synthétise le référentiel spatial, le parc technique, les associations historisées par intervalles temporels, le flux des mesures IoT et le sous-système d'alertes et d'interventions :



3.2 Entités et identifiants

Entité	Identifiant conceptuel	Attributs métier principaux	Unicité métier candidate
Campus	`id_campus`	code, nom, adresse, statut	`code_campus`
Bâtiment	`id_batiment`	code, nom, adresse, superficie, statut	`code_batiment` dans le périmètre retenu
Zone	`id_zone`	code, nom, type, étage, statut	`(bâtiment, code_zone)`
Équipement	`id_equipement`	code, libellé, type, fabricant, référence	`code_equipement`; numéro de série

Entité	Identifiant conceptuel	Attributs métier principaux	Unicité métier candidate
		constructeur, numéro de série, date de mise en service, statut	éventuellement unique si le métier le garantit
Historique_Zone_Equipement	`id_historique_zone`	équipement, zone, début, fin, motif	Intervalles non chevauchants pour un équipement
Capteur	`id_capteur`	identifiant externe, type, libellé, unité nominale, statut, date de mise en service	`identifiant_externe` dans son périmètre
Affectation_Capteur_Equipement	`id_affectation_capteur`	capteur, équipement, début, fin, motif	Intervalles non chevauchants pour un capteur
Mesure	`id_mesure`	capteur, horodatage événement/réception, valeur, unité, qualité, clé de déduplication éventuelle	Identifiant technique; la déduplication dépend du contrat source
Règle_Alerte	`id_regle_alerte`	code/version, portée, indicateur, opérateur, seuil, sévérité, validité, statut	`(code_regle, numero_version)`
Alerte	`id_alerte`	règle, cible, sévérité constatée, statut, dates d'acquittement/clôture, commentaire	Identifiant technique
Intervention	`id_intervention`	référence, équipement, demandeur optionnel, type, priorité, statut, dates, description, compte rendu	`reference`
Utilisateur	`id_utilisateur`	courriel, prénom, nom, statut, date de création	`courriel`
Technicien	`id_technicien`	utilisateur associé, matricule, spécialité, statut	`matricule`; un compte au plus par profil technicien
Affectation_Technicien_Intervention	`id_affectation_technicien`	intervention, technicien, début/fin d'affectation, rôle, commentaire	Identifiant d'occurrence; règles de chevauchement à préciser
Lien_Alerte_Intervention	`id_lien_alerte_intervention`	alerte, intervention, date de liaison, commentaire	`(id_alerte, id_intervention)`

Réconciliation avec le sujet du TD : le sujet pédagogique cite notamment la superficie du bâtiment, la référence constructeur, la date d'installation du capteur et les nom/prénom/courriel/spécialité du technicien. Ils sont intégrés ici comme données métier

: `superficie_m2` est une mesure de surface, `reference_constructeur` complète les informations du modèle, la date d'installation correspond à la mise en service du capteur, et les données de personne sont portées par `Utilisateur` (la spécialité reste dans le profil `Technicien`). La précision et l'unicité de ces champs doivent être validées par rapport aux formulaires et au vocabulaire métier.

3.3 Associations et cardinalités

La cardinalité est indiquée au voisinage de chaque entité, comme minimum et maximum de participations.

Association	Cardinalités	Sens et règle de gestion
Contenir (Campus–Bâtiment)	Campus `(1,N)` — Bâtiment `(1,1)`	Un campus contient un ou plusieurs bâtiments; chaque bâtiment relève d'un campus unique. Le minimum côté campus est une hypothèse à confirmer si l'on autorise la création d'un campus vide.
Contenir (Bâtiment–Zone)	Bâtiment `(1,N)` — Zone `(1,1)`	Un bâtiment contient au moins une zone selon le modèle de travail; chaque zone dépend d'un bâtiment unique.
Localiser / Historique_Zone_Equipement (Équipement–Zone)	Équipement `(0,N)` — Zone `(0,N)` au fil du temps; chaque occurrence de l'association relie exactement `(1,1)` équipement et `(1,1)` zone	Une occurrence représente un intervalle de localisation. Un équipement ne possède pas deux localisations simultanées; une zone peut accueillir plusieurs équipements.
Affecter / Affectation_Capteur_Equipement (Capteur–Équipement)	Capteur `(0,N)` — Équipement `(0,N)` au fil du temps; chaque occurrence relie exactement `(1,1)` capteur et `(1,1)` équipement	Historique de remplacement/réaffectation; un capteur n'est relié qu'à un équipement au même instant; un équipement peut avoir plusieurs capteurs.
Produire (Capteur–Mesure)	Capteur `(0,N)` — Mesure `(1,1)`	Chaque mesure est issue d'un capteur; un capteur peut ne pas avoir encore émis ou avoir émis plusieurs mesures.
Déclencher (Règle_Alerte–Alerte)	Règle `(0,N)` — Alerte `(1,1)`	Toute alerte référence la version exacte de la règle appliquée.
Concerner (Équipement–Alerte)	Équipement `(0,N)` — Alerte `(0,1)`	Première branche de la cible d'alerte.
Concerner (Capteur–Alerte)	Capteur `(0,N)` — Alerte `(0,1)`	Seconde branche; avec la précédente, contrainte XOR : exactement une cible par alerte.
Être à l'origine de (Mesure–Alerte)	Mesure `(0,N)` — Alerte `(0,1)`	Une alerte peut être corrélée à une mesure précise ou à une évaluation agrégée sans mesure source unique.
Faire l'objet de (Équipement–Intervention)	Équipement `(0,N)` — Intervention `(1,1)`	Chaque intervention cible un équipement dans le périmètre retenu; un équipement peut connaître plusieurs interventions.
Demander (Utilisateur–Intervention)	Utilisateur `(0,N)` — Intervention `(0,1)`	Le demandeur peut être absent pour une intervention préventive ou automatique.

Association	Cardinalités	Sens et règle de gestion
Être le compte de (Utilisateur–Technicien)	Utilisateur `(0,1)` — Technicien `(1,1)`	Chaque profil technicien est rattaché à un compte; un utilisateur n'a au plus qu'un profil technicien.
Participer / Affectation_Technicien_Intervention	Technicien `(0,N)` — Intervention `(0,N)` via l'association	Plusieurs techniciens peuvent participer à une intervention; l'association conserve rôle et période.
Relier / Lien_Alerte_Intervention	Alerte `(0,N)` — Intervention `(0,N)` via l'association	Permet plusieurs alertes pour une intervention et plusieurs interventions liées à une alerte, avec date de liaison.

3.4 Dictionnaire de données synthétique

Les types ci-dessous sont conceptuels. **Obligatoire** signifie que l'élément est requis pour une occurrence valide selon le modèle de travail; les dates de fin et autres champs indiqués facultatifs peuvent être absents. Les clés étrangères conceptuelles sont listées dans les associations et ne sont pas répétées comme attributs métier des entités.

Référentiel et identités

Entité / association	Attribut	Définition métier / type	Identifiant ?	Obligatoire ?
Campus	`id_campus`	Entier 64 bits, identifiant technique stable	Oui	Oui
Campus	`code_campus`	Chaîne (80), code métier du campus	Non	Oui
Campus	`nom`	Chaîne (160), nom usuel	Non	Oui
Campus	`adresse`	Chaîne (500), localisation ou adresse	Non	Non
Campus	`statut`	Chaîne contrôlée (30), état du campus	Non	Oui
Bâtiment	`id_batiment`	Entier 64 bits, identifiant technique stable	Oui	Oui
Bâtiment	`code_batiment`	Chaîne (80), code métier	Non	Oui
Bâtiment	`nom`	Chaîne (160), nom usuel	Non	Oui
Bâtiment	`adresse`	Chaîne (500), adresse ou accès	Non	Non
Bâtiment	`superficie_m2`	Décimal en m², surface de référence	Non	Non, à confirmer
Bâtiment	`statut`	Chaîne contrôlée (30), état du référentiel	Non	Oui
Zone	`id_zone`	Entier 64 bits, identifiant technique stable	Oui	Oui
Zone	`code_zone`	Chaîne (80), code unique dans le bâtiment	Non	Oui
Zone	`nom`	Chaîne (160), nom affiché	Non	Oui
Zone	`type_zone`	Chaîne contrôlée (50), salle, étage, local technique, etc.	Non	Oui
Zone	`etage`	Chaîne (40), niveau du bâtiment	Non	Non
Zone	`statut`	Chaîne contrôlée (30), état de la zone	Non	Oui

Entité / association	Attribut	Définition métier / type	Identifiant ?	Obligatoire ?
Équipement	id_equipement	Entier 64 bits, identifiant technique stable	Oui	Oui
Équipement	code_equipement	Chaîne (100), référence interne	Non	Oui
Équipement	libelle	Chaîne (200), désignation lisible	Non	Oui
Équipement	type_equipement	Chaîne contrôlée (80), famille fonctionnelle	Non	Oui
Équipement	marque	Chaîne (100), fabricant/marque	Non	Non
Équipement	reference_constructeur	Chaîne (120), référence fournie par le constructeur	Non	Non
Équipement	modele	Chaîne (100), modèle déclaré	Non	Non
Équipement	numero_serie	Chaîne (120), série fabricant	Non	Non
Équipement	date_mise_en_service	Date, entrée en exploitation de l'équipement	Non	Non
Équipement	statut	Chaîne contrôlée (30), état du parc	Non	Oui
Utilisateur	id_utilisateur	Entier 64 bits, identifiant technique	Oui	Oui
Utilisateur	courriel	Chaîne (254), adresse de contact/compte	Non	Oui
Utilisateur	prenom	Chaîne (100), prénom	Non	Oui
Utilisateur	nom	Chaîne (100), nom	Non	Oui
Utilisateur	statut	Chaîne contrôlée (30), état du compte	Non	Oui
Utilisateur	date_creation	Horodatage UTC, création du compte	Non	Oui
Technicien	id_technicien	Entier 64 bits, identifiant du profil	Oui	Oui
Technicien	matricule	Chaîne (80), référence professionnelle	Non	Oui
Technicien	specialite	Chaîne contrôlée (100), domaine de compétence	Non	Non
Technicien	statut	Chaîne contrôlée (30), état du profil	Non	Oui

Historisation, IoT et alertes

Entité / association	Attribut	Définition métier / type	Identifiant ?	Obligatoire ?
Historique_Zone_Equipement	id_historique_zone	Entier 64 bits, occurrence d'historique	Oui	Oui
Historique_Zone_Equipement	date_debut	Horodatage UTC, début inclusif de localisation	Non	Oui
Historique_Zone_Equipement	date_fin	Horodatage UTC, fin exclusive; absente pour période ouverte	Non	Non
Historique_Zone_Equipement	motif_deplacement	Chaîne (500), motif de déplacement/fin	Non	Non

Entité / association	Attribut	Définition métier / type	Identifiant ?	Obligatoire ?
Capteur	id_capteur	Entier 64 bits, identifiant interne	Oui	Oui
Capteur	identifiant_externe	Chaîne (160), identité transmise par source/passerelle	Non	Oui
Capteur	type_capteur	Chaîne contrôlée (80), grandeur/famille	Non	Oui
Capteur	libelle	Chaîne (200), nom affiché	Non	Oui
Capteur	unite_nominale	Chaîne (30), unité attendue	Non	Oui
Capteur	statut	Chaîne contrôlée (30), état opérationnel	Non	Oui
Capteur	date_mise_en_service	Date, mise en service/installation	Non	Non
Affectation_Capteur_Equipement	id_affectation_capteur	Entier 64 bits, période de rattachement	Oui	Oui
Affectation_Capteur_Equipement	date_debut	Horodatage UTC, début inclusif d'affectation	Non	Oui
Affectation_Capteur_Equipement	date_fin	Horodatage UTC, fin exclusive; absente si courante	Non	Non
Affectation_Capteur_Equipement	motif_affectation	Chaîne (500), installation/réaffectation/retrait	Non	Non
Mesure	id_mesure	Entier 64 bits, identifiant technique conceptuel	Oui	Oui
Mesure	horodatage_evenement	Horodatage UTC, instant de l'observation	Non	Oui
Mesure	horodatage_reception	Horodatage UTC, instant d'arrivée plateforme	Non	Oui
Mesure	valeur	Décimal haute précision, valeur observée	Non	Oui
Mesure	unite	Chaîne (30), unité portée par cette observation	Non	Oui
Mesure	qualite	Chaîne contrôlée (30), validité/qualité de la donnée	Non	Oui
Mesure	cle_deduplication	Chaîne (160), identifiant source s'il existe	Non	Non
Règle_Alerte	id_regle_alerte	Entier 64 bits, identifiant de version	Oui	Oui
Règle_Alerte	code_regle	Chaîne (100), code stable de la règle logique	Non	Oui
Règle_Alerte	numero_version	Entier, version de la règle	Non	Oui

Entité / association	Attribut	Définition métier / type	Identifiant ?	Obligatoire ?
Règle_Alerte	`libelle`	Chaîne (200), nom de règle	Non	Oui
Règle_Alerte	`portee_cible`	Enum `EQUIPEMENT` / `CAPTEUR`, cible attendue	Non	Oui
Règle_Alerte	`indicateur`	Chaîne (80), grandeur évaluée	Non	Oui
Règle_Alerte	`operateur`	Chaîne contrôlée (20), comparaison au seuil	Non	Oui
Règle_Alerte	`seuil`	Décimal haute précision, seuil configuré	Non	Oui
Règle_Alerte	`severite`	Chaîne contrôlée (30), gravité configurée	Non	Oui
Règle_Alerte	`date_debut_validite`	Horodatage UTC, début inclusif d'application	Non	Oui
Règle_Alerte	`date_fin_validite`	Horodatage UTC, fin exclusive de validité	Non	Non
Règle_Alerte	`statut`	Chaîne contrôlée (30), brouillon/publiée/retirée selon référentiel	Non	Oui
Alerte	`id_alerte`	Entier 64 bits, identifiant technique	Oui	Oui
Alerte	`date_creation`	Horodatage UTC, création de l'occurrence	Non	Oui
Alerte	`severite_constatee`	Chaîne contrôlée (30), sévérité de l'occurrence	Non	Oui
Alerte	`statut`	Chaîne contrôlée (30), état de traitement	Non	Oui
Alerte	`date_acquittement`	Horodatage UTC, acquittement éventuel	Non	Non
Alerte	`date_cloture`	Horodatage UTC, clôture éventuelle	Non	Non
Alerte	`commentaire`	Texte, contexte de suivi	Non	Non

Interventions et associations porteuses d'attributs

Entité / association	Attribut	Définition métier / type	Identifiant ?	Obligatoire ?
Intervention	`id_intervention`	Entier 64 bits, identifiant technique	Oui	Oui
Intervention	`reference`	Chaîne (100), référence lisible de suivi	Non	Oui

Entité / association	Attribut	Définition métier / type	Identifiant ?	Obligatoire ?
Intervention	type_intervention	Chaîne contrôlée (30), nature de l'action	Non	Oui
Intervention	priorite	Chaîne contrôlée (30), niveau de priorité	Non	Oui
Intervention	statut	Chaîne contrôlée (30), état de cycle de vie	Non	Oui
Intervention	date_ouverture	Horodatage UTC, création/ouverture	Non	Oui
Intervention	date_planifiee	Horodatage UTC, date prévue	Non	Non
Intervention	date_debut	Horodatage UTC, début effectif	Non	Non
Intervention	date_cloture	Horodatage UTC, fin effective	Non	Non
Intervention	description	Texte, objet/symptôme de l'intervention	Non	Oui
Intervention	compte_rendu	Texte, travail et résultat constaté	Non	Non
Affectation_Technicien_Intervention	id_affectation_technicien	Entier 64 bits, occurrence d'affectation	Oui	Oui
Affectation_Technicien_Intervention	date_affectation	Horodatage UTC, début inclusif de participation	Non	Oui
Affectation_Technicien_Intervention	date_fin_affectation	Horodatage UTC, fin exclusive	Non	Non
Affectation_Technicien_Intervention	role_dans_intervention	Chaîne contrôlée (40), rôle de participation	Non	Non
Affectation_Technicien_Intervention	commentaire	Texte, contexte de l'affectation	Non	Non
Lien_Alerte_Intervention	id_lien_alerte_intervention	Entier 64 bits, identifiant du lien	Oui	Oui
Lien_Alerte_Intervention	date_liaison	Horodatage UTC, création du lien	Non	Oui
Lien_Alerte_Intervention	commentaire	Texte, motif/contexte du lien	Non	Non

Les attributs d'association simples sont les identifiants des entités reliées : campus dans bâtiment, bâtiment dans zone, utilisateur dans technicien, etc. Ils deviendront des clés étrangères au MLD, sans être considérés comme des identifiants métier indépendants.

3.5 Historisation : localisation et affectation des capteurs

Les deux associations temporelles sont représentées par des périodes **semi-ouvertes** `[début, fin[`, exprimées en UTC. Une période ouverte a une borne de fin absente.

- **Localisation équipement–zone** : chaque déplacement ferme l'occurrence précédente et ouvre une nouvelle occurrence. Le lieu courant est celui dont l'intervalle contient l'instant présent; le lieu passé est retrouvé avec l'intervalle qui contient la date de l'événement. Les intervalles d'un même équipement ne doivent pas se chevaucher.
- **Affectation capteur–équipement** : la relation n'est pas une FK permanente du capteur vers l'équipement. Le remplacement ou déplacement clôt l'affectation existante et en crée une nouvelle. Les périodes d'un même capteur ne se chevauchent pas, mais plusieurs capteurs peuvent être affectés simultanément au même équipement.
- **Mesures** : restent rattachées à leur capteur d'origine. Pour interpréter une mesure, l'affectation et la localisation sont résolues à `horodatage_evenement`, pas à l'état courant.

Une FK garantit l'existence des références, mais ne garantit pas à elle seule que la mesure soit comprise dans une période d'affectation ni que la zone de l'équipement soit compatible avec l'analyse effectuée. Cette cohérence temporelle est à appliquer par le service métier ou par des contrôles dédiés.

4. Étape 3 — Modèle logique et physique PostgreSQL

4.1 MLD relationnel

Notation : `PK` clé primaire, `FK` clé étrangère, `UQ` unicité, `NN` non nul.

```
CAMPUS(PK id_campus, UQ code_campus, nom, adresse, statut)
BATIMENT(PK id_batiment, FK id_campus -> CAMPUS, UQ code_batiment,
         nom, adresse, superficie_m2, statut)
ZONE(PK id_zone, FK id_batiment -> BATIMENT,
     UQ (id_batiment, code_zone), nom, type_zone, etage, statut)

EQUIPEMENT(PK id_equipement, UQ code_equipement, libelle, type_equipement,
           marque, reference_constructeur, modele, numero_serie,
           date_mise_en_service, statut)
HISTORIQUE_ZONE_EQUIPEMENT(PK id_historique_zone,
  FK id_equipement -> EQUIPEMENT, FK id_zone -> ZONE,
  date_debut, date_fin, motif_deplacement)

CAPTEUR(PK id_capteur, UQ identifiant_externe, type_capteur, libelle,
        unite_nominale, statut, date_mise_en_service)
AFFECTATION_CAPTEUR_EQUIPEMENT(PK id_affectation_capteur,
  FK id_capteur -> CAPTEUR, FK id_equipement -> EQUIPEMENT,
  date_debut, date_fin, motif_affectation)
MESURE(PK physique (horodatage_evenement, id_mesure),
       FK id_capteur -> CAPTEUR, horodatage_reception, valeur, unite,
       qualite, cle_deduplication)

REGLE_ALERTE(PK id_regle_alerte, UQ (code_regle, numero_version),
  libelle, portee_cible, indicateur, operateur, seuil, severite,
  date_debut_validite, date_fin_validite, statut)
ALERTE(PK id_alerte, FK id_regle_alerte -> REGLE_ALERTE,
  FK id_equipement_cible -> EQUIPEMENT NULL,
  FK id_capteur_cible -> CAPTEUR NULL,
  FK (horodatage_mesure_declencheuse, id_mesure_declencheuse) -> MESURE NULL,
  date_creation, severite_constatee, statut, date_acquittement,
  date_cloture, commentaire, CHECK cible équipement XOR capteur)
```

```
UTILISATEUR(PK id_utilisateur, UQ courriel, prenom, nom, statut, date_creation)
TECHNICIEN(PK id_technicien, FK UQ id_utilisateur -> UTILISATEUR,
           UQ matricule, specialite, statut)
INTERVENTION(PK id_intervention, UQ reference,
  FK id_equipement -> EQUIPEMENT, FK id_demandeur -> UTILISATEUR NULL,
  type_intervention, priorite, statut, date_ouverture, date_planifiee,
  date_debut, date_cloture, description, compte_rendu)
AFFECTATION_TECHNICIEN_INTERVENTION(PK id_affectation_technicien,
  FK id_intervention -> INTERVENTION, FK id_technicien -> TECHNICIEN,
  date_affectation, date_fin_affectation, role_dans_intervention, commentaire)
LIEN_ALERTE_INTERVENTION(PK id_lien_alerte_intervention,
  FK id_alerte -> ALERTE, FK id_intervention -> INTERVENTION,
  UQ (id_alerte, id_intervention), date_liaison, commentaire)
```

Le MLD retient un identifiant conceptuel `id_mesure`, mais PostgreSQL exige que la clé unique d'une table partitionnée inclue sa clé de partition. La PK physique comprend donc aussi `horodatage_evenement`; une référence relationnelle à une mesure porte les deux champs. Le couple `(id_capteur, horodatage_evenement)` n'est pas déclaré unique, car des observations distinctes peuvent partager un instant et la stratégie de déduplication reste à définir.

4.2 MPD : types, contraintes et index

Besoin	Choix PostgreSQL	Motif
Identifiants	`BIGINT GENERATED BY DEFAULT AS IDENTITY`	Marge de capacité adaptée aux tables volumineuses.
Instants	`TIMESTAMPTZ`	Comparaison cohérente des instants; stocker/interpréter en UTC.
Dates civiles	`DATE`	Mise en service sans heure lorsque cela suffit.
Codes et libellés	`VARCHAR(n)`	Bornes explicites et contrôles métier.
Commentaires / comptes rendus	`TEXT`	Longueur variable.
Valeurs et seuils	`NUMERIC(20,6)` comme point de départ	Précision décimale, à confirmer par grandeur et unité.
États / unités	`VARCHAR(n)` + `CHECK` ou tables de référence	Vocabulaire contrôlé; tables de référence si administration/évolution requise.
Périodes	`tstzrange(début, fin, '[)')` dans des exclusions GiST	Interdire les chevauchements tout en autorisant des périodes contiguës.

Contraintes majeures :

- clés étrangères `NOT NULL` pour les associations obligatoires; références nullables pour les liens facultatifs;
- `CHECK (date_fin IS NULL OR date_debut < date_fin)` pour les intervalles;
- exclusions GiST sur `(id_equipement, période)` pour l'historique de zone et `(id_capteur, période)` pour l'affectation;
- contrainte `CHECK` d'exclusivité et d'existence d'une cible unique pour l'alerte; contrôle additionnel que la portée de la règle correspond à la cible;
- unicités métier sur codes, références, courriel, matricule et version de règle selon les périmètres retenus;
- le cycle de vie des statuts et les dates corrélées (ex. clôture) demandent des règles métier supplémentaires; aucune contrainte non confirmée n'est supposée ici.

Indexation initiale à évaluer avec `EXPLAIN (ANALYZE, BUFFERS)` :

Index / ordre de colonnes	Usage
Mesures `(id_capteur, horodatage_evenement DESC)` avec colonnes de lecture en `INCLUDE`	Dernière mesure et fenêtre historique par capteur.
Mesures `(horodatage_evenement)`	Filtrage temporel global dans les partitions.
Historique `(id_equipement, date_debut DESC)` et `(id_zone)`	Position courante/passée et équipements d'une zone.
Affectation capteur `(id_capteur, date_debut DESC)` et `(id_equipement)`	Rattachement à une date et capteurs d'un équipement.
Alertes par cible, statut, date; index partiels par cible	Listes d'alertes actives par capteur ou équipement.
Interventions `(id_equipement, statut, date_ouverture DESC)` et `(statut, date_ouverture DESC)`	Files de maintenance et historique d'un équipement.
Clés étrangères côté enfant	Jointures et opérations référentielles; PostgreSQL ne les indexe pas automatiquement.

Chaque index améliore certains parcours mais renchérit insertions, stockage et maintenance. Les index sont à conserver uniquement s'ils sont utiles aux requêtes observées.

4.3 Extrait DDL de référence

Le code est un **extrait documentaire PostgreSQL 15+**, pas un script complet déclaré exécuté. Il illustre les types et contraintes structurantes. Les partitions mensuelles doivent être créées avant l'ingestion et entretenues par une procédure supervisée.

```
CREATE SCHEMA IF NOT EXISTS smartcampus;
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE smartcampus.campus (
    id_campus BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code_campus VARCHAR(80) NOT NULL UNIQUE,
    nom VARCHAR(160) NOT NULL,
    adresse VARCHAR(500),
    statut VARCHAR(30) NOT NULL
);

CREATE TABLE smartcampus.batiment (
    id_batiment BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_campus BIGINT NOT NULL REFERENCES smartcampus.campus(id_campus),
    code_batiment VARCHAR(80) NOT NULL UNIQUE,
    nom VARCHAR(160) NOT NULL,
    adresse VARCHAR(500),
    superficie_m2 NUMERIC(12,2) CHECK (superficie_m2 IS NULL OR superficie_m2 > 0),
    statut VARCHAR(30) NOT NULL
);

CREATE TABLE smartcampus.zone (
    id_zone BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_batiment BIGINT NOT NULL REFERENCES smartcampus.batiment(id_batiment),
    code_zone VARCHAR(80) NOT NULL,
    nom VARCHAR(160) NOT NULL,
    type_zone VARCHAR(50) NOT NULL,
    etage VARCHAR(40),
    statut VARCHAR(30) NOT NULL,
    UNIQUE (id_batiment, code_zone)
);
```

```sql
CREATE TABLE smartcampus.equipement (
    id_equipement BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code_equipement VARCHAR(100) NOT NULL UNIQUE,
    libelle VARCHAR(200) NOT NULL,
    type_equipement VARCHAR(80) NOT NULL,
    marque VARCHAR(100),
    reference_constructeur VARCHAR(120),
    modele VARCHAR(100),
    numero_serie VARCHAR(120),
    date_mise_en_service DATE,
    statut VARCHAR(30) NOT NULL
);

CREATE TABLE smartcampus.historique_zone_equipement (
    id_historique_zone BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_equipement BIGINT NOT NULL
        REFERENCES smartcampus.equipement(id_equipement),
    id_zone BIGINT NOT NULL REFERENCES smartcampus.zone(id_zone),
    date_debut TIMESTAMPTZ NOT NULL,
    date_fin TIMESTAMPTZ,
    motif_deplacement VARCHAR(500),
    CHECK (date_fin IS NULL OR date_debut < date_fin),
    EXCLUDE USING gist (
        id_equipement WITH =,
        tstzrange(date_debut, date_fin, '[)') WITH &&
    )
);

CREATE TABLE smartcampus.capteur (
    id_capteur BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    identifiant_externe VARCHAR(160) NOT NULL UNIQUE,
    type_capteur VARCHAR(80) NOT NULL,
    libelle VARCHAR(200) NOT NULL,
    unite_nominale VARCHAR(30) NOT NULL,
    statut VARCHAR(30) NOT NULL,
    date_mise_en_service DATE
);

CREATE TABLE smartcampus.affectation_capteur_equipement (
    id_affectation_capteur BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_capteur BIGINT NOT NULL REFERENCES smartcampus.capteur(id_capteur),
    id_equipement BIGINT NOT NULL
        REFERENCES smartcampus.equipement(id_equipement),
    date_debut TIMESTAMPTZ NOT NULL,
    date_fin TIMESTAMPTZ,
    motif_affectation VARCHAR(500),
    CHECK (date_fin IS NULL OR date_debut < date_fin),
    EXCLUDE USING gist (
        id_capteur WITH =,
        tstzrange(date_debut, date_fin, '[)') WITH &&
    )
);

CREATE TABLE smartcampus.mesure (
    id_mesure BIGINT GENERATED BY DEFAULT AS IDENTITY,
    id_capteur BIGINT NOT NULL REFERENCES smartcampus.capteur(id_capteur),
    horodatage_evenement TIMESTAMPTZ NOT NULL,
    horodatage_reception TIMESTAMPTZ NOT NULL,
    valeur NUMERIC(20,6) NOT NULL,
    unite VARCHAR(30) NOT NULL,
    qualite VARCHAR(30) NOT NULL,
    cle_deduplication VARCHAR(160),
```

```
    PRIMARY KEY (horodatage_evenement, id_mesure)
) PARTITION BY RANGE (horodatage_evenement);

CREATE TABLE smartcampus.mesure_2025_01
    PARTITION OF smartcampus.mesure
    FOR VALUES FROM ('2025-01-01 00:00:00+00')
                TO ('2025-02-01 00:00:00+00');

CREATE INDEX ix_mesure_capteur_evenement
    ON smartcampus.mesure (id_capteur, horodatage_evenement DESC)
    INCLUDE (valeur, unite, qualite, horodatage_reception);

CREATE TABLE smartcampus.utilisateur (
    id_utilisateur BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    courriel       VARCHAR(254) NOT NULL UNIQUE,
    prenom         VARCHAR(100) NOT NULL,
    nom            VARCHAR(100) NOT NULL,
    statut         VARCHAR(30)  NOT NULL
        CHECK (statut IN ('ACTIF', 'SUSPENDU', 'DESACTIVE')),
    date_creation  TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE smartcampus.technicien (
    id_technicien  BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_utilisateur BIGINT       NOT NULL UNIQUE
        REFERENCES smartcampus.utilisateur (id_utilisateur) ON UPDATE RESTRICT ON DELETE
RESTRICT,
    matricule      VARCHAR(80)  NOT NULL UNIQUE,
    specialite     VARCHAR(100),
    statut         VARCHAR(30)  NOT NULL
        CHECK (statut IN ('ACTIF', 'INDISPONIBLE', 'INACTIF'))
);

CREATE TABLE smartcampus.regle_alerte (
    id_regle_alerte      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code_regle           VARCHAR(100) NOT NULL,
    numero_version       INTEGER      NOT NULL CHECK (numero_version > 0),
    libelle              VARCHAR(200) NOT NULL,
    portee_cible         VARCHAR(20)  NOT NULL
        CHECK (portee_cible IN ('EQUIPEMENT', 'CAPTEUR')),
    indicateur           VARCHAR(80)  NOT NULL,
    operateur            VARCHAR(20)  NOT NULL
        CHECK (operateur IN ('>', '>=', '<', '<=', '=', '<>')),
    seuil                NUMERIC(20, 6) NOT NULL,
    severite             VARCHAR(30)  NOT NULL
        CHECK (severite IN ('INFO', 'MINEURE', 'MAJEURE', 'CRITIQUE')),
    date_debut_validite  TIMESTAMPTZ  NOT NULL,
    date_fin_validite    TIMESTAMPTZ,
    statut               VARCHAR(20)  NOT NULL
        CHECK (statut IN ('BROUILLON', 'ACTIVE', 'SUSPENDUE', 'EXPIREE')),
    CONSTRAINT uq_regle_alerte_version UNIQUE (code_regle, numero_version),
    CONSTRAINT ck_regle_alerte_validite
        CHECK (date_fin_validite IS NULL OR date_debut_validite < date_fin_validite)
);

CREATE TABLE smartcampus.alerte (
    id_alerte                    BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_regle_alerte              BIGINT      NOT NULL
        REFERENCES smartcampus.regle_alerte (id_regle_alerte) ON UPDATE RESTRICT ON DELETE
RESTRICT,
    id_equipement_cible          BIGINT
        REFERENCES smartcampus.equipement (id_equipement) ON UPDATE RESTRICT ON DELETE RESTRICT,
```

```
    id_capteur_cible                BIGINT
        REFERENCES smartcampus.capteur (id_capteur) ON UPDATE RESTRICT ON DELETE RESTRICT,
    id_mesure_declencheuse          BIGINT,
    horodatage_mesure_declencheuse  TIMESTAMPTZ,
    date_creation                   TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    severite_constatee              VARCHAR(30) NOT NULL
        CHECK (severite_constatee IN ('INFO', 'MINEURE', 'MAJEURE', 'CRITIQUE')),
    statut                          VARCHAR(30) NOT NULL
        CHECK (statut IN ('OUVERTE', 'ACQUITTEE', 'EN_COURS', 'CLOTUREE', 'IGNOREE')),
    date_acquittement               TIMESTAMPTZ,
    date_cloture                    TIMESTAMPTZ,
    commentaire                     TEXT,
    CONSTRAINT ck_alerte_cible_xor
        CHECK ((id_equipement_cible IS NOT NULL) <> (id_capteur_cible IS NOT NULL)),
    CONSTRAINT ck_alerte_mesure_declencheuse
        CHECK (
            (id_mesure_declencheuse IS NULL AND horodatage_mesure_declencheuse IS NULL)
            OR
            (id_mesure_declencheuse IS NOT NULL AND horodatage_mesure_declencheuse IS NOT NULL)
        ),
    CONSTRAINT fk_alerte_mesure_declencheuse
        FOREIGN KEY (horodatage_mesure_declencheuse, id_mesure_declencheuse)
        REFERENCES smartcampus.mesure (horodatage_evenement, id_mesure)
        ON UPDATE RESTRICT ON DELETE RESTRICT
);

CREATE TABLE smartcampus.intervention (
    id_intervention       BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    reference             VARCHAR(100) NOT NULL UNIQUE,
    id_equipement         BIGINT       NOT NULL
        REFERENCES smartcampus.equipement (id_equipement) ON UPDATE RESTRICT ON DELETE RESTRICT,
    id_demandeur          BIGINT
        REFERENCES smartcampus.utilisateur (id_utilisateur) ON UPDATE RESTRICT ON DELETE
RESTRICT,
    type_intervention     VARCHAR(30)  NOT NULL
        CHECK (type_intervention IN ('CORRECTIVE', 'PREVENTIVE', 'INSPECTION', 'AUTRE')),
    priorite              VARCHAR(30)  NOT NULL
        CHECK (priorite IN ('BASSE', 'NORMALE', 'HAUTE', 'URGENTE')),
    statut                VARCHAR(30)  NOT NULL
        CHECK (statut IN (
            'A_PLANIFIER', 'PLANIFIEE', 'EN_COURS', 'EN_ATTENTE', 'TERMINEE', 'ANNULEE'
        )),
    date_ouverture        TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    date_planifiee        TIMESTAMPTZ,
    date_debut            TIMESTAMPTZ,
    date_cloture          TIMESTAMPTZ,
    description           TEXT        NOT NULL,
    compte_rendu          TEXT
);

CREATE TABLE smartcampus.affectation_technicien_intervention (
    id_affectation_technicien BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_intervention           BIGINT      NOT NULL
        REFERENCES smartcampus.intervention (id_intervention) ON UPDATE RESTRICT ON DELETE
RESTRICT,
    id_technicien             BIGINT      NOT NULL
        REFERENCES smartcampus.technicien (id_technicien) ON UPDATE RESTRICT ON DELETE RESTRICT,
    date_affectation          TIMESTAMPTZ NOT NULL,
    date_fin_affectation      TIMESTAMPTZ,
    role_dans_intervention    VARCHAR(40),
    commentaire               TEXT,
    CONSTRAINT ck_ati_intervalle
```

```
        CHECK (
            date_fin_affectation IS NULL
            OR date_affectation < date_fin_affectation
        )
);

CREATE TABLE smartcampus.lien_alerte_intervention (
    id_lien_alerte_intervention BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_alerte                   BIGINT      NOT NULL
        REFERENCES smartcampus.alerte (id_alerte) ON UPDATE RESTRICT ON DELETE RESTRICT,
    id_intervention             BIGINT      NOT NULL
        REFERENCES smartcampus.intervention (id_intervention) ON UPDATE RESTRICT ON DELETE
RESTRICT,
    date_liaison                TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    commentaire                 TEXT,
    CONSTRAINT uq_lien_alerte_intervention UNIQUE (id_alerte, id_intervention)
);

-- Trigger de validation d'adéquation entre la portée de la règle et la cible de l'alerte
CREATE OR REPLACE FUNCTION smartcampus.verifier_cible_alerte()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    portee_attendue VARCHAR(20);
BEGIN
    SELECT r.portee_cible
      INTO portee_attendue
      FROM smartcampus.regle_alerte AS r
     WHERE r.id_regle_alerte = NEW.id_regle_alerte;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Regle d''alerte % inexistante', NEW.id_regle_alerte
            USING ERRCODE = 'foreign_key_violation';
    END IF;

    IF portee_attendue = 'EQUIPEMENT'
       AND (NEW.id_equipement_cible IS NULL OR NEW.id_capteur_cible IS NOT NULL) THEN
        RAISE EXCEPTION 'La regle % attend une cible equipement', NEW.id_regle_alerte
            USING ERRCODE = 'check_violation';
    ELSIF portee_attendue = 'CAPTEUR'
       AND (NEW.id_capteur_cible IS NULL OR NEW.id_equipement_cible IS NOT NULL) THEN
        RAISE EXCEPTION 'La regle % attend une cible capteur', NEW.id_regle_alerte
            USING ERRCODE = 'check_violation';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_alerte_cible_portee
BEFORE INSERT OR UPDATE OF id_regle_alerte, id_equipement_cible, id_capteur_cible
ON smartcampus.alerte
FOR EACH ROW
EXECUTE FUNCTION smartcampus.verifier_cible_alerte();

-- Index complémentaires pour les jointures et filtres d'exploitation
CREATE INDEX ix_batiment_id_campus ON smartcampus.batiment (id_campus);
CREATE INDEX ix_zone_id_batiment ON smartcampus.zone (id_batiment);
CREATE INDEX ix_hze_zone ON smartcampus.historique_zone_equipement (id_zone);
CREATE INDEX ix_hze_equipement_debut ON smartcampus.historique_zone_equipement (id_equipement,
date_debut DESC);
```

```
CREATE INDEX ix_ace_equipement ON smartcampus.affectation_capteur_equipement (id_equipement);
CREATE INDEX ix_ace_capteur_debut ON smartcampus.affectation_capteur_equipement (id_capteur,
date_debut DESC);

CREATE INDEX ix_alerte_equipement_etat_date
    ON smartcampus.alerte (id_equipement_cible, statut, date_creation DESC)
    WHERE id_equipement_cible IS NOT NULL;
CREATE INDEX ix_alerte_capteur_etat_date
    ON smartcampus.alerte (id_capteur_cible, statut, date_creation DESC)
    WHERE id_capteur_cible IS NOT NULL;
CREATE INDEX ix_alerte_regle_date
    ON smartcampus.alerte (id_regle_alerte, date_creation DESC);

CREATE INDEX ix_intervention_equipement_statut_date
    ON smartcampus.intervention (id_equipement, statut, date_ouverture DESC);
CREATE INDEX ix_intervention_statut_date
    ON smartcampus.intervention (statut, date_ouverture DESC);
CREATE INDEX ix_ati_technicien
    ON smartcampus.affectation_technicien_intervention (id_technicien, date_affectation DESC);
CREATE INDEX ix_lai_intervention
    ON smartcampus.lien_alerte_intervention (id_intervention);
```

Ce script DDL complet matérialise l'ensemble des entités du périmètre métier, leurs types précis, leurs contraintes d'intégrité relationnelle et temporelle (exclusions GiST), le partitionnement de la table de mesures à forte volumétrie et les déclencheurs procéduraux d'intégrité croisée. Avant déploiement en production, les partitions mensuelles ultérieures sont créées par un job d'exploitation planifié et le script est intégré à l'outil de migration (ex. Flyway ou Liquibase).

5. Étape 4 — Normalisation et dénormalisations maîtrisées

5.1 Normalisation

Le référentiel est conçu en **3NF**, et généralement en **BCNF sous les dépendances fonctionnelles retenues** : chaque entité porte ses attributs propres, les identifiants métier sont contrôlés, et les relations plusieurs-à-plusieurs ou historisées sont représentées séparément. Exemple : le nom du campus n'est pas répété dans les zones; la localisation n'est pas stockée comme un seul attribut mutable de l'équipement; l'unité nominale du capteur n'élimine pas l'unité effectivement portée par chaque mesure.

Cette conclusion repose sur les dépendances et règles supposées ici; elle devra être réévaluée si des règles telles que « une référence constructeur détermine un modèle » ou « une zone est identifiée globalement » sont confirmées.

La séparation des données répond à des besoins différents :

- **Historique de localisation et affectation** : préserver les périodes et éviter l'écrasement de l'état antérieur.
- **Mesures** : stocker des observations append-only à haute cadence, avec horodatage d'événement et de réception.
- **Référentiel** : garantir unicité, cohérence des relations et mises à jour contrôlées.
- **Alertes/interventions** : conserver les événements et leur cycle de vie sans confondre règle, occurrence et action humaine.

5.2 Redondances et coûts de lecture

Les jointures et scans peuvent devenir coûteux pour : obtenir la dernière valeur de chacun des capteurs, calculer un état synthétique d'équipement, produire des agrégats de consommation ou réunir alertes/interventions/localisation. Le besoin de rafraîchissement du tableau de bord toutes les cinq minutes motive une mesure de ces requêtes, mais ne prouve pas à lui seul qu'une dénormalisation est nécessaire.

5.3 Projections candidates

Projection	Données dérivées	Bénéfice attendu	Coût / cohérence
`etat_courant_capteur`	Dernière mesure reçue et dernière mesure valide,	Lecture rapide d'environ 600	Mise à jour atomique ou par outbox; une mesure tardive ne doit pas écraser une

Projection	Données dérivées	Bénéfice attendu	Coût / cohérence
	avec horodatage et qualité	états	valeur plus récente selon l'ordre métier retenu. Reconstructible depuis les mesures conservées.
État courant d'équipement	Santé calculée depuis capteurs, règles et fraîcheur	Éviter de recalculer la synthèse pour chaque tableau de bord	Doit exposer la règle de calcul et sa date; statut dérivé, jamais autorité métier indépendante.
Agrégats horaires/journaliers	Sommes, moyennes, minima/maxima par période et périmètre	Réduire les scans des rapports répétés	Définir unités, qualité, données tardives et corrections; recalculer les fenêtres affectées.

La démarche recommandée est d'abord d'optimiser les requêtes et index PostgreSQL. Une projection n'est introduite que si un benchmark reproduit montre un bénéfice nécessaire. La mise à jour passe par la même transaction d'ingestion lorsque possible, ou par une outbox idempotente; un cache externe éventuel est une copie reconstruisible, jamais une seconde source de vérité.

6. Étape 5 — Comparaison NoSQL et sélection

Paradigme	Forces	Limites au regard de SmartCampus	Usage possible
Clé-valeur	Accès direct par clé, lecture rapide d'un état ponctuel	Relations, historiques et agrégations limités; crée un état dérivé à synchroniser	Cache éventuel de dernière valeur/ état courant
Document	Schéma souple, stockage d'objets autonomes imbriqués	Contraintes référentielles, relations et mises à jour multi-objets moins naturelles	Configurations réellement hétérogènes; `JSONB` PostgreSQL suffit potentiellement
Colonne / wide-column	Écritures distribuées, séries conçues autour d'une clé de partition	Modèle dirigé par les requêtes, relations/transactions limitées, exploitation additionnelle	Réexaminer si le benchmark prouve les limites durables du stockage de séries PostgreSQL
Graphe	Traversées multi-sauts et relations explicites	Coût d'un moteur supplémentaire; complexité potentiellement faible à l'échelle actuelle	Dépendances complexes et parcours dynamiques s'ils deviennent fréquents

6.1 Décision

Aucun moteur NoSQL n'est retenu pour la première version. Le référentiel est petit, relationnel et temporel; le débit nominal est de l'ordre de 20 mesures/s, avec un historique volumineux mais partitionnable. Le nombre de lignes seul ne suffit pas à justifier un moteur distribué.

PostgreSQL conserve la source de vérité. Les seules options conditionnelles sont :

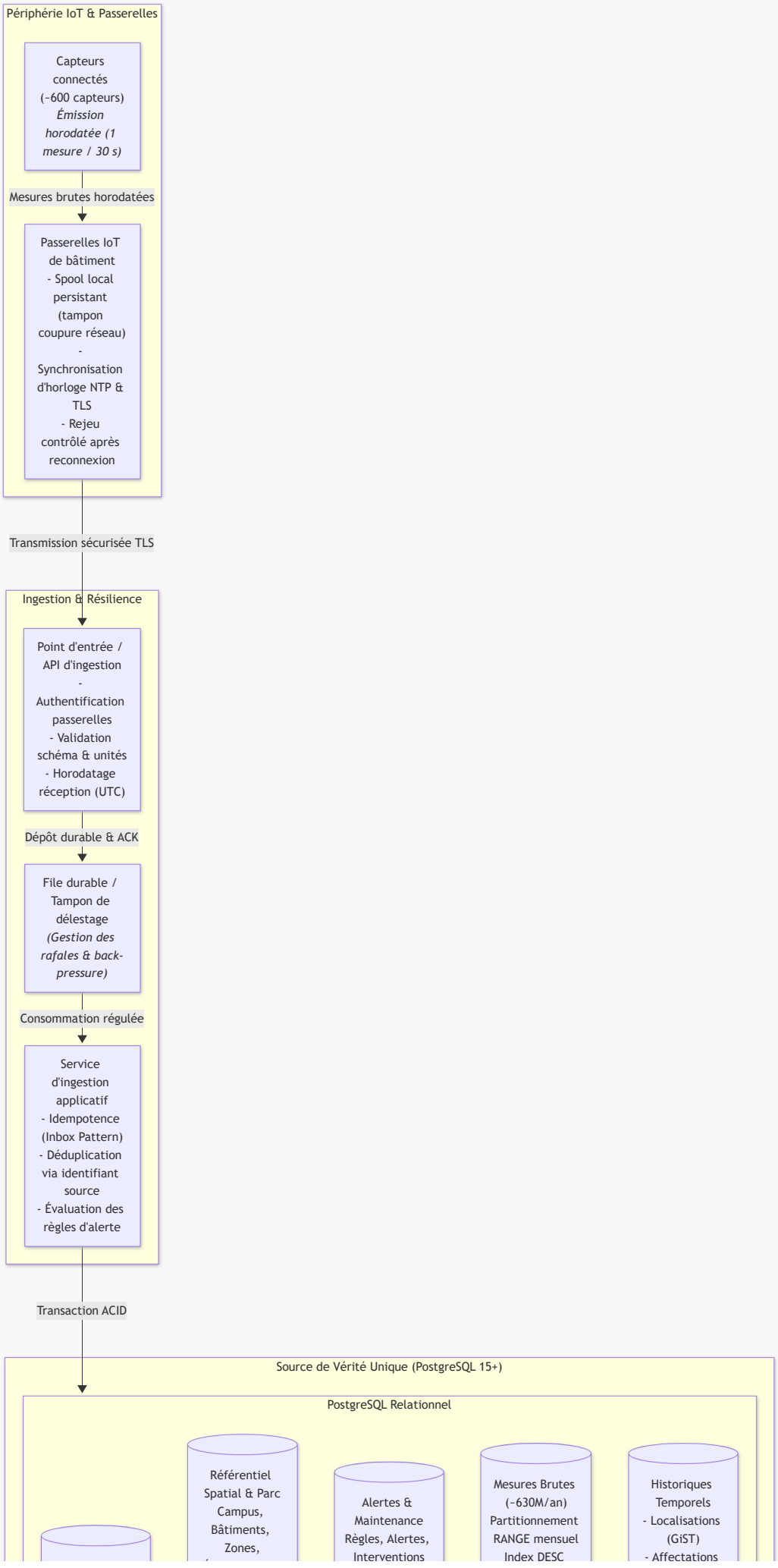
- cache clé-valeur pour états courants si le benchmark démontre un SLO de lecture non atteint;
- moteur wide-column si l'ingestion, la capacité ou les coûts des séries deviennent durablement insatisfaisants après optimisation;
- moteur analytique si scans historiques et concurrence décisionnelle dépassent les capacités acceptables;
- graphe si les parcours de dépendances multi-sauts deviennent une fonction centrale et coûteuse en relationnel.

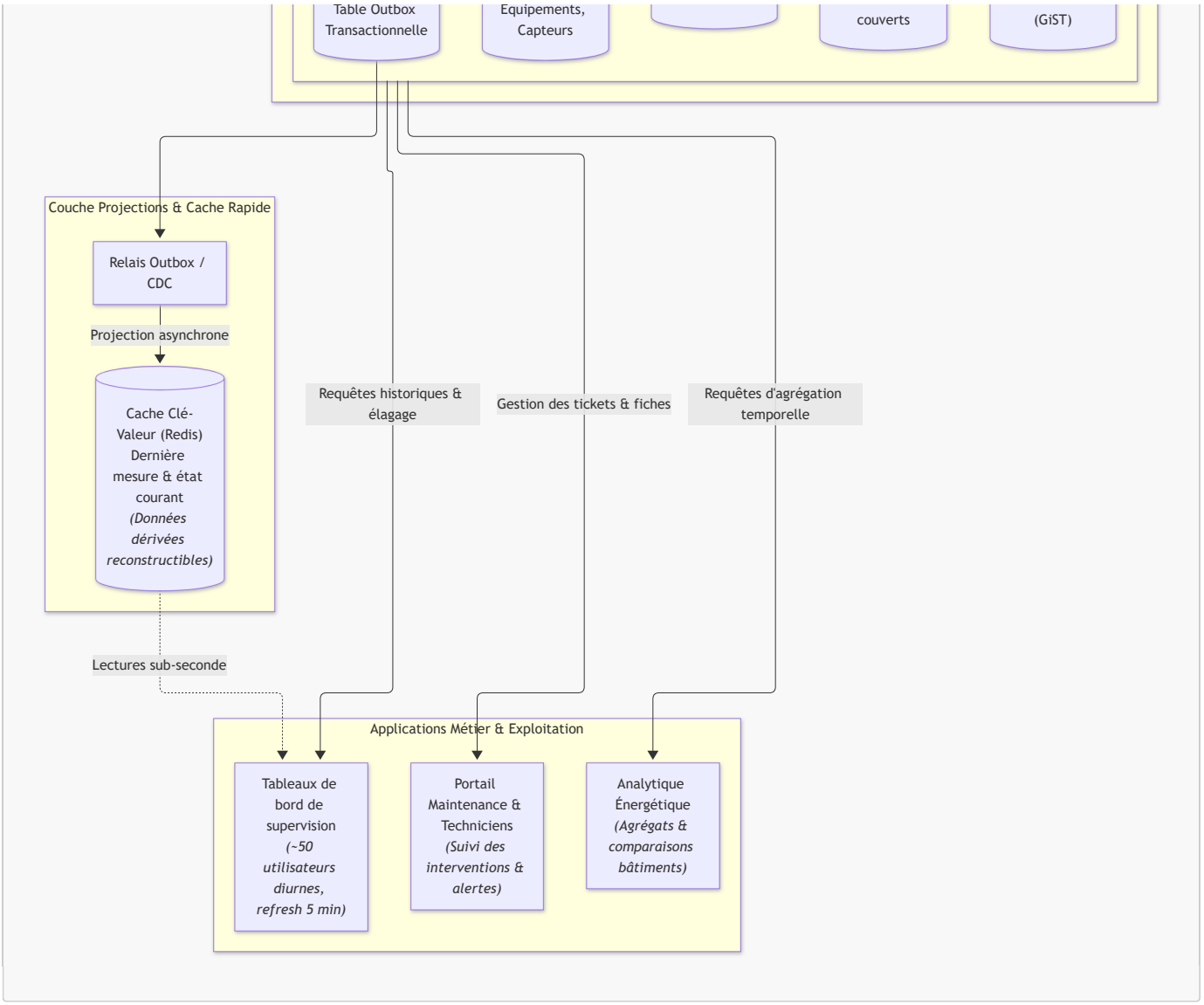
Chaque ajout exigerait un flux de synchronisation, une politique de cohérence, un plan de reconstruction, une supervision et une évaluation du coût complet.

7. Étape 6 — Architecture, flux et résilience

7.1 Architecture logique et flux de bout en bout

Le diagramme ci-dessous illustre l'architecture globale des flux, depuis les capteurs physiques jusqu'aux applications de supervision, en intégrant les mécanismes de résilience (spool passerelle, tampon d'ingestion, partitionnement et découplage par table Outbox) :


Périphérie IoT & Passerelles
Capteurs connectés (~600 capteurs) Émission horodatée (1 mesure / 30 s)
Mesures brutes horodatées
Passerelles IoT de bâtiment - Spool local persistant (tampon coupure réseau) - Synchronisation d'horloge NTP & TLS - Rejeu contrôlé après reconnexion
Transmission sécurisée TLS
Ingestion & Résilience
Point d'entrée / API d'ingestion - Authentification passerelles - Validation schéma & unités - Horodatage réception (UTC)
Dépôt durable & ACK
File durable / Tampon de délestage (Gestion des rafales & back-pressure)
Consommation régulée
Service d'ingestion applicatif - Idempotence (Inbox Pattern) - Déduplication via identifiant source - Évaluation des règles d'alerte
Transaction ACID
Source de Vérité Unique (PostgreSQL 15+)
PostgreSQL Relationnel
Référentiel Spatial & Parc Campus, Bâtiments, Zones,
Alertes & Maintenance Règles, Alertes, Interventions
Mesures Brutes (~630M/an) Partitionnement RANGE mensuel Index DESC
Historiques Temporels - Localisations (GiST) - Affectations



PostgreSQL porte le référentiel, les mesures brutes, les règles, alertes, interventions et associations historisées. La file durable et le cache ne sont pas une source de vérité. Le cache externe n'est pas requis au démarrage.

7.2 Flux métier

1. **Mesure** : la passerelle émet un message identifié; l'entrée authentifie et valide le format; l'ingestion enregistre l'événement et sa réception, applique la déduplication et confirme uniquement après la persistance définie.
2. **Alerte** : les règles versionnées évaluent une mesure ou une fenêtre; la création d'alerte et son événement d'outbox sont transactionnels; l'idempotence empêche de créer plusieurs occurrences lors d'un rejeu selon une clé de décision à définir.
3. **Intervention** : un utilisateur ou processus crée l'intervention; les affectations de techniciens et liens aux alertes sont historisés et audités.
4. **Consultation** : l'API sert tableaux de bord et historiques à partir de PostgreSQL, d'index/projections éventuels et d'agrégats explicitement maintenus.

7.3 Synchronisation, idempotence et données tardives

- Traiter en **au moins une fois**, avec consommateurs idempotents; ne pas prétendre à un « exactement une fois » de bout en bout sans contrat et preuve.
- Utiliser un identifiant de message fourni par la source et une portée d'unicité, éventuellement associé à un identifiant de capteur/passerelle. Si ce contrat n'existe pas, documenter explicitement la limite de déduplication.
- Enregistrer séparément événement et réception; conserver les événements tardifs et appliquer les règles selon l'heure événement, sans les rejeter uniquement parce qu'ils sont anciens.
- Une projection d'état courant n'est mise à jour que si l'événement satisfait l'ordre choisi (heure événement, séquence source ou autre règle déterministe). Distinguer dernière reçue et dernière valide.

- Utiliser une inbox pour le suivi des messages reçus/traités/quarantainés et une outbox transactionnelle pour publier les changements après commit.
- Réconcilier périodiquement les comptes, doublons, retards, états courants et agrégats; reconstruire les projections depuis PostgreSQL.

7.4 Coupures, RPO/RTO et reprise

En cas de perte réseau, la passerelle conserve les mesures dans un spool durable, surveille sa capacité puis les retransmet au rétablissement du lien. L'espace doit être dimensionné selon durée de coupure, taille réelle des messages, cadence et marge. À la cadence nominale, une journée représente déjà 1 728 000 messages. Une politique de saturation doit être explicite; ne jamais supprimer silencieusement les événements les plus anciens.

Scénario	Proposition de travail, non approuvée	Préconditions
Processus applicatif/consommateur indisponible	RPO 0 pour les transactions validées; reprise ≤ 15 min à évaluer	Rejeu durable, inbox/outbox, redémarrage supervisé
Panne du primaire PostgreSQL sur site	RPO 0 seulement si réplication synchrone et bascule testées; sinon dépend du retard mesuré. RTO ≤ 15 min comme cible de travail	Réplica, fencing, tests de bascule et d'intégrité
Perte du site	RPO ≤ 5 min et RTO ≤ 4 h comme hypothèses à valider	Réplication/sauvegarde hors site, WAL, infrastructure de secours et exercices
Panne réseau d'une passerelle	Pas de perte des messages encore persistés dans le spool; borne dépend de sa capacité et de sa propre disponibilité	Capacité de tampon, alertes et rejeu idempotent

Ces chiffres ne sont ni SLA ni garanties. La réplication synchrone augmente potentiellement la latence et lie la disponibilité au secondaire; l'asynchrone accepte une perte de transactions récentes au sinistre. Sauvegardes complètes, archivage WAL/PITR, copies hors site, restauration réelle et procédures de bascule sont requis indépendamment de la réplication.

8. Étape 7 — Protocole de benchmark reproductible

8.1 Principe et environnement

Aucun résultat de benchmark n'a été exécuté ou n'est revendiqué dans ce dossier. Les nombres de charge sont des paramètres calculés ou scénarios, pas des observations.

Chaque campagne doit consigner : date/opérateur, versions PostgreSQL/extensions/OS, commit applicatif et générateur, CPU/mémoire/disques, réseau, topologie, réglages PostgreSQL, schéma/index/partitions, état des caches, outils de charge et instrumentation. L'environnement doit être isolé, représentatif de la cible et contrôlé pour éviter que le générateur ne constitue le goulot d'étranglement.

8.2 Dataset

- Construire les 8 bâtiments, environ 160 zones, 250 équipements et 600 capteurs, avec une distribution déterministe documentée et une graine fixe.
- Peupler les associations historisées sans chevauchements; documenter la distribution des règles, alertes, interventions, techniciens et statuts.
- Générer les mesures de façon déterministe : horodatage événement/réception, valeur, unité, qualité, doublons et retards contrôlés. Les plages métier de valeurs doivent venir d'un profil défini, non inventé.
- Tester des paliers de 10 M, 100 M et 630 720 000 mesures si les moyens le permettent. Ces paliers valident des échelles différentes; un palier réduit ne prouve pas la capacité annuelle.
- Vérifier le nombre par partition/capteur/période, les contraintes, les trous/doublons intentionnels et conserver un manifeste avec seed, paramètres, compte attendu et empreintes.

8.3 Scénarios de charge

Scénario	Charge proposée	Ce qu'il faut observer
Ingestion nominale	20 messages/s sur une durée documentée	Débit offert/accepté/inséré, retards, erreurs et croissance du WAL
Rafale synchronisée	600 messages simultanés toutes les 30 s	Latence et backlog de pointe, pertes/rejets et durée de retour à la normale
Reconnexion passerelle	Backlog de durée paramétrée, envoi avec limite définie	Temps de rattrapage, équité entre sources et effet sur l'ingestion nominale
Tableaux de bord	50 utilisateurs concurrents; rafraîchissement toutes les 5 min, y compris pic synchronisé	Latence API/DB, débit, saturation et fraîcheur des données
Historique	Requêtes par capteur et fenêtres temporelles, avec lignes et partitions touchées consignées	Temps de réponse, taille des résultats, ressources et effet du pruning
Alertes	Taux de règles/événements et proportion déclenchante explicités	Latence événement→alerte, nombre attendu/observé, doublons, erreurs et backlog

Exécuter séparément lecture seule, écriture seule et charge mixte. L'ingestion applicative réelle doit être testée; un chargement bulk contournant contraintes ou validations n'est pas substituable au flux temps réel.

8.4 Métriques, répétitions et limites

Mesurer débit offert/accepté/inséré, latence bout en bout événement→affichage (p50/p95/p99), latence de dernière valeur et d'historique, latence de traitement des alertes, taux d'erreur/rejet/doublon, CPU, mémoire, I/O, espace/index/WAL, connexions, retard de file et backlog.

Réaliser idéalement au moins cinq répétitions par scénario depuis un état initial restauré. Consigner cache froid/chaud séparément, médiane et dispersion, écarts de configuration, perturbations et essais invalidés avec motif. Synchroniser les horloges et définir le point de mesure de la latence.

Avant le test, les parties prenantes doivent définir le débit minimal, les seuils p95/p99, la fraîcheur admise, le taux d'erreur acceptable, les plafonds de ressources, le rattrapage et les objectifs RPO/RTO. Sans critères d'acceptation, le benchmark décrit des résultats mais ne peut pas déclarer la solution conforme.

Limites : un dataset synthétique ne garantit pas la représentativité des valeurs et corrélations réelles; les mesures base seule ne qualifient pas le chemin complet; un test à 50 utilisateurs n'a de sens qu'avec les parcours et requêtes décrits; des paliers partiels ne remplacent pas le test annuel; aucun chiffre de latence ou capacité n'est extrapolé sans exécution.

9. Étape 8 — Recommandations, risques et évolutions

9.1 Synthèse des choix

Domaine	Recommandation
Source de vérité	PostgreSQL seul au démarrage
Modèle	Référentiel normalisé; associations historiques explicites
Mesures	Partitionnement mensuel par `horodatage_evenement`, rétention brute cible de 12 mois
Index	Ciblés sur capteur/temps, clés étrangères et recherches d'état
État courant	Commencer par les index/requêtes; projection SQL si le besoin est démontré
Ingestion	Tampons durables, accusé après persistance, rejeu et idempotence
NoSQL	Aucun moteur imposé; cache clé-valeur seulement si benchmark concluant

Domaine	Recommandation
Résilience	Sauvegarde/PITR, exercices de restauration, RPO/RTO à faire approuver
Performance	Pas de résultats fictifs; qualification par protocole reproductible

9.2 Alternatives écartées

- **Wide-column comme stockage natif des mesures dès la première version** : complexité distribuée et cohérence supplémentaires sans preuve que PostgreSQL partitionné soit insuffisant.
- **Clé-valeur comme source de vérité** : inadéquation aux relations, contraintes, historiques et analyses.
- **Base documentaire autonome pour le référentiel** : structure et intégrité relationnelles prédominantes; des métadonnées variables pourraient être envisagées en `JSONB`.
- **Base graphe immédiate** : le parc et les parcours connus sont modestes; les jointures temporelles relationnelles suffisent au périmètre actuel.
- **Dénormalisation massive** : augmente les risques de divergence et le coût de correction; seules les projections reconstructibles répondant à un besoin mesuré sont admises.
- **Absence d'historisation** : rendrait incorrecte la restitution de la zone ou de l'équipement associés à une observation passée.
- **Réplication synchrone sans validation de SLA** : choix de disponibilité/latence/coût impossible à fixer sans objectifs approuvés et essais de bascule.

9.3 Risques et mesures de maîtrise

Risque	Effet possible	Mesure de maîtrise
Goulot d'étranglement d'ingestion ou d'indexation	Retard, backlog et impact sur les lectures	Mesurer charge réelle, limiter index inutiles, dimensionner lots/connexions, tester rafales
Dérive des horloges / événements désordonnés	Mauvaise attribution temporelle, état courant erroné	NTP/diagnostic, heure réception séparée, séquence source si disponible, indicateurs de dérive
Identifiant de message absent ou non stable	Doublons lors des reprises	Contrat d'idempotence, clé et périmètre source, tests de rejeu; ne pas dédupliquer par hypothèse
FK d'alerte vers mesure conservée dans une partition ancienne	Purge de partition bloquée après 12 mois	Arbitrer rétention des preuves; instantané d'audit, archivage ou référence sans FK après stratégie définie
Partitions manquantes ou purge erronée	Échec d'ingestion ou perte de données	Création anticipée, monitoring de couverture, purge idempotente et auditée, essais de restauration
Données tardives après agrégation	Indicateurs et projections incomplets	Recalcul des fenêtres affectées et suivi de fraîcheur/backlog
Dépendance excessive à la réplication	Faux sentiment de sauvegarde ou perte en sinistre	Sauvegardes hors site, PITR, tests réels de restauration et procédures PRA
Règles d'unité/qualité non stabilisées	Analyses incohérentes et alertes erronées	Contrat par type de capteur, conversions explicites et corpus de vérification

9.4 Trajectoire d'évolution

1. Valider les règles métier ouvertes, les unités, la qualité, la déduplication, les cycles d'alerte et les durées de conservation.
2. Finaliser le schéma et exécuter les migrations sur PostgreSQL cible; automatiser les partitions et les contrôles d'intégrité.
3. Définir les contrats d'ingestion, taille des tampons, accusés, rejeu, quarantaines et procédures de réconciliation.
4. Définir SLA, RPO/RTO et critères d'acceptation; conduire le benchmark et tester restauration/bascule.
5. Ajouter des projections SQL, agrégats ou cache seulement sur déclencheur mesuré.

6. Réexaminer wide-column, analytique dédié ou graphe si croissance et benchmarks démontrent une limite concrète.
7. Étendre au multi-campus en précisant périmètres d'accès, unicité des codes, délégation et éventuelle séparation logique.

Conclusion

L'architecture SmartCampus doit d'abord garantir l'exactitude temporelle et la traçabilité du métier : un équipement n'est pas toujours dans la même zone, un capteur peut être remplacé, une mesure peut arriver en retard, et une alerte doit rester interprétable dans le contexte où elle a été produite. Le MCD et le modèle relationnel rendent ces faits explicites plutôt que de les réduire à un état courant mutable.

PostgreSQL constitue un socle cohérent pour le périmètre étudié, sous réserve de confirmer le modèle avec les règles métier et d'éprouver le MPD sur des volumes représentatifs. Le partitionnement, les index ciblés et les tampons durables répondent aux contraintes identifiées sans imposer d'emblée plusieurs moteurs. La valeur de cette architecture ne dépend pas seulement du choix du SGBD : elle dépend tout autant de l'identité des messages, de la qualité des horloges, de la politique de rétention, du cycle de vie des partitions et de la capacité à restaurer réellement les données.

La décision d'ajouter un cache, un moteur analytique ou un stockage distribué doit donc rester conditionnelle à des objectifs approuvés et à des mesures reproductibles. Cette progression — clarifier, construire, mesurer, puis faire évoluer — maintient une source de vérité claire, limite les coûts opérationnels et laisse ouverte une croissance vers davantage de campus, de capteurs et d'usages analytiques.

---

10. Annexe — Requêtes SQL d'exploitation et scénarios de charge

Cette annexe rassemble les requêtes critiques du système de données, conçues pour valider les performances d'accès, l'élagage des partitions (*partition pruning*), la cohérence spatio-temporelle et le respect des objectifs de latence des tableaux de bord.

10.1 Requête d'état courant pour les tableaux de bord (Supervision diurne)

Cette requête extrait la dernière observation valide pour l'ensemble des ~600 capteurs du campus. Elle s'exécute typiquement toutes les 5 minutes pour rafraîchir la vue d'ensemble des 50 utilisateurs connectés, en s'appuyant sur l'index couvrant `ix_mesure_capteur_evenement`.

```sql
-- Récupération de la dernière mesure valide par capteur
SELECT DISTINCT ON (m.id_capteur)
    m.id_capteur,
    c.identifiant_externe,
    c.type_capteur,
    m.id_mesure,
    m.horodatage_evenement,
    m.horodatage_reception,
    m.valeur,
    m.unite,
    m.qualite
FROM smartcampus.mesure AS m
JOIN smartcampus.capteur AS c ON c.id_capteur = m.id_capteur
WHERE m.qualite = 'VALIDE'
  AND m.horodatage_evenement >= CURRENT_TIMESTAMP - INTERVAL '15 minutes'
ORDER BY m.id_capteur, m.horodatage_evenement DESC;
```

Note d'optimisation : La clause temporelle `horodatage_evenement >= CURRENT_TIMESTAMP - INTERVAL '15 minutes'` permet à PostgreSQL de ne scanner que la partition courante du mois, évitant un balayage de l'historique complet des 630 millions de lignes. Si le temps d'exécution dépasse le seuil cible sous forte concurrence, cette requête alimente la projection matérialisée `etat_courant_capteur` synchronisée par Outbox.

10.2 Requête d'historique temporel avec élagage de partition (*Partition Pruning*)

Cette requête extrait la série chronologique d'un capteur cible sur une fenêtre de 7 jours pour un tracé graphique sur le poste de l'exploitant.

```
-- Diagnostic historique d'un capteur sur une fenêtre temporelle
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    id_mesure,
    horodatage_evenement,
    valeur,
    unite,
    qualite
FROM smartcampus.mesure
WHERE id_capteur = :id_capteur_cible
  AND horodatage_evenement >= '2025-03-01 00:00:00+00'
  AND horodatage_evenement <  '2025-03-08 00:00:00+00'
ORDER BY horodatage_evenement ASC;
```

Comportement attendu : Grâce au partitionnement par `RANGE (horodatage_evenement)`, le planificateur PostgreSQL applique l'élagage statique et dynamique (*Partition Pruning*), en ignorant totalement 11 des 12 partitions mensuelles et en restreignant la recherche à l'index local de la partition concernée.

10.3 Résolution spatio-temporelle : localisation et équipement à date d'événement

Pour diagnostiquer une défaillance ou qualifier une alerte, cette requête reconstitue l'environnement physique complet (équipement supervisé, zone et bâtiment) en vigueur exactement au moment où l'observation a été mesurée, sans être faussée par les déplacements ou réaffectations ultérieurs.

```
-- Reconstitution du contexte physique exact d'une observation passée
SELECT
    m.id_mesure,
    m.horodatage_evenement,
    m.valeur,
    m.unite,
    c.id_capteur,
    c.identifiant_externe AS code_capteur,
    e.id_equipement,
    e.code_equipement,
    e.libelle AS nom_equipement,
    z.id_zone,
    z.code_zone,
    z.nom AS nom_zone,
    b.code_batiment,
    b.nom AS nom_batiment
FROM smartcampus.mesure AS m
JOIN smartcampus.capteur AS c
    ON c.id_capteur = m.id_capteur
-- 1. Résolution de l'équipement rattaché au capteur à la date de l'événement
JOIN smartcampus.affectation_capteur_equipement AS ace
    ON ace.id_capteur = m.id_capteur
   AND m.horodatage_evenement >= ace.date_debut
   AND (ace.date_fin IS NULL OR m.horodatage_evenement < ace.date_fin)
JOIN smartcampus.equipement AS e
    ON e.id_equipement = ace.id_equipement
-- 2. Résolution de la zone où se trouvait l'équipement à la date de l'événement
JOIN smartcampus.historique_zone_equipement AS hze
    ON hze.id_equipement = e.id_equipement
   AND m.horodatage_evenement >= hze.date_debut
   AND (hze.date_fin IS NULL OR m.horodatage_evenement < hze.date_fin)
```

```
JOIN smartcampus.zone AS z
    ON z.id_zone = hze.id_zone
JOIN smartcampus.batiment AS b
    ON b.id_batiment = z.id_batiment
WHERE m.id_mesure = :id_mesure_cible
  AND m.horodatage_evenement = :horodatage_mesure;
```

10.4 Agrégation de consommation énergétique par bâtiment (Pas de 5 minutes)

Cette requête permet au responsable énergie d'analyser les profils de consommation électrique ou thermique par bâtiment, agrégés par fenêtres régulières de 5 minutes.

```
-- Calcul de la puissance moyenne ou cumul par pas de 5 minutes et par bâtiment
SELECT
    b.code_batiment,
    b.nom AS batiment,
    date_bin(INTERVAL '5 minutes', m.horodatage_evenement, TIMESTAMPTZ '2025-01-01 00:00:00+00')
AS creneau_5min,
    COUNT(m.id_mesure) AS nb_mesures,
    ROUND(AVG(m.valeur), 2) AS valeur_moyenne,
    m.unite
FROM smartcampus.mesure AS m
JOIN smartcampus.affectation_capteur_equipement AS ace
    ON ace.id_capteur = m.id_capteur
   AND m.horodatage_evenement >= ace.date_debut
   AND (ace.date_fin IS NULL OR m.horodatage_evenement < ace.date_fin)
JOIN smartcampus.historique_zone_equipement AS hze
    ON hze.id_equipement = ace.id_equipement
   AND m.horodatage_evenement >= hze.date_debut
   AND (hze.date_fin IS NULL OR m.horodatage_evenement < hze.date_fin)
JOIN smartcampus.zone AS z ON z.id_zone = hze.id_zone
JOIN smartcampus.batiment AS b ON b.id_batiment = z.id_batiment
WHERE m.qualite = 'VALIDE'
  AND m.horodatage_evenement >= '2025-03-01 08:00:00+00'
  AND m.horodatage_evenement <  '2025-03-01 12:00:00+00'
GROUP BY b.code_batiment, b.nom, creneau_5min, m.unite
ORDER BY b.code_batiment, creneau_5min;
```